IMY 220

Assignment 4

React and Asynchronous API Requests

Due: 30 September at 13:00

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

This is an individual, take-home assignment.

This assignment focuses on:

- retrieving data from an Express API;
- using `fetch()` with `async` and `await`;
- using `useEffect()` to retrieve data when a React application loads;
- sending data to an Express API;
- using different HTTP request methods;
- updating React state after asynchronous operations;
- using controlled forms and user input;
- passing data and functions between React components;
- filtering data stored in React state;
- loading and error handling.

In this assignment, you will work with a supplied Express API and complete a React application that communicates with the API.

The Express server is responsible for providing and managing the post data.

The React application is responsible for displaying the data and allowing the user to interact with it.

Scenario

You are continuing your work on the PhotoShare social media platform.

PhotoShare can already retrieve and display posts from an API.

The application now requires additional functionality that allows users to interact with the posts without refreshing the page.

Another developer has created the basic React and Express project structure.

Your task is to complete the application so that PhotoShare users can:

- retrieve posts from the Express API;
- display the retrieved posts;
- add new posts;
- delete existing posts;
- search the loaded posts;
- manually refresh the posts;
- receive appropriate loading and error feedback.

All communication between the React application and Express server must take place asynchronously.

Supplied Files

Download the supplied project from ClickUP.

The project contains two main folders:

assignment-4/

├── client/

└── server/

The `client` folder contains the React application.

The `server` folder contains the supplied Express API.

Do not substantially change the supplied project structure unless instructed.

Replace uXXXXXXX with your student number where indicated.

Ensure that your student number is included as a comment in all JavaScript and JSX files that you modify.

Client

The supplied React application contains the basic project structure required for the PhotoShare application.

A stylesheet has been supplied. You may modify or extend the supplied styling if required, but styling is not the primary focus of this assignment.

Install the required dependencies using:

npm install

Run the React application using:

npm run dev

Keep the client running while completing and testing the assignment.

Server

The supplied Express application contains the API and initial PhotoShare post data.

Install the required dependencies using:

npm install

Start the Express server using:

npm start

The server will run on:

`http://localhost:3001`

Do not change the supplied server port.

Keep the server running while completing and testing the assignment.

The API uses an in-memory array to store posts.

This means that changes made to the posts will remain available while the Express server is running, but the original supplied data will be restored when the server is restarted.

Supplied API

The Express server provides the following API endpoints:

Method	Endpoint	Purpose
GET	`/api/posts`	Retrieve all posts
POST	`/api/posts`	Add a new post
DELETE	`/api/posts/:id`	Delete a post

Each post follows the following structure:

```
{
   id: 1,
   username: "@sarah",
   caption: "Studying React today!"
}
```

The server is responsible for generating the `id` of a newly created post.

Do not generate a temporary post ID inside the React application.

Section 1: Load Posts in React

Inside the main React application, create the state required to store:

- posts;
- loading status;
- error messages.

When the React application first loads, retrieve the posts from:

GET /api/posts

Create an asynchronous function responsible for retrieving the posts.

Use:

- `fetch()`;
- `async` and `await`;
- `response.ok`;
- `response.json()`.

Use `useEffect()` to call the function when the component first loads.

The effect must not repeatedly request the posts unnecessarily.

When the posts have been retrieved successfully:

1. Convert the response to JSON.
2. Store the returned posts in React state.
3. Update the loading state appropriately.

Section 2: Display the Posts

Create reusable React components to display the posts.

Your application must include:

App

├── AddPostForm

├── SearchPosts

└── PostList
    └── Post

You may create additional components if required.

Pass the posts from `App` to `PostList` using props.

Inside `PostList`, use `map()` to create a reusable `Post` component for each post.

Each post must display:

- the username;
- the caption;
- a Delete button.

Use the post `id` as the React key.

For example:

key={post.id}

Do not use the array index as the key.

If there are no posts to display, show:

No posts found.

Section 3: Add a Post

Create a component called:

AddPostForm

The form must contain:

- a Username input;
- a Caption input;
- an Add Post button.

Both inputs must be controlled using React state.

When the form is submitted:

1. Prevent the page from refreshing.
2. Validate that both fields contain a value.
3. If either value is empty, do not send the request.
4. Send a `POST` request to `/api/posts`.
5. Send the username and caption as JSON in the request body.
6. Include the appropriate request headers.
7. Wait for the server response.
8. Check whether the response was successful.
9. Convert the returned response to JSON.
10. Add the post returned by the server to the posts stored in React state.
11. Clear the form fields.

The page must not be refreshed to display the newly added post.

Use the post returned by the Express server because it contains the `id` generated by the server.

Section 4: Delete a Post

Each `Post` component must contain a button displaying:

Delete

When the button is clicked, send a `DELETE` request to:

/api/posts/:id

Replace `:id` with the ID of the selected post.

For example, deleting the post with an ID of 3 must send a request to:

DELETE /api/posts/3

The delete functionality must:

1. Receive the ID of the selected post.
2. Send the appropriate `DELETE` request.
3. Wait for the server response.
4. Check whether the response was successful.
5. Remove the deleted post from React state after the server confirms that the deletion was successful.

Use `filter()` to create the updated posts array.

The page must not refresh when a post is deleted.

Pass the required delete functionality to the appropriate components using props.

Section 5: Search Posts

Create a component called:

SearchPosts

The component must contain a controlled text input.

The user must be able to search posts using:

- username;
- caption.

The search must:

- be case-insensitive;
- update while the user types;
- display all posts when the search field is empty;
- not send an additional request to the Express server.

Use the posts already stored in React state to perform the search.

When no posts match the current search, display:

No posts found.

Section 6: Loading and Error Handling

The application must provide feedback while asynchronous operations are taking place.

While the initial posts are being retrieved, display:

Loading posts...

The posts must not be displayed while the initial request is loading.

Check `response.ok` where appropriate when communicating with the API.

Use `try` and `catch` to handle errors caused by asynchronous API operations.

If the posts cannot be retrieved, display:

Unable to load posts.

If a post cannot be added, display:

Unable to add post.

If a post cannot be deleted, display:

Unable to delete post.

Error messages must be displayed in the application interface and not only in the browser console.

The application must remain usable after an error occurs.

Section 7: Refresh Posts

Add a button displaying:

Refresh Posts

When the button is clicked:

1. Set the loading state appropriately.
2. Clear any existing error.
3. Retrieve the posts from the Express API again.

Display the loading message while the posts are being retrieved.

The page must not be refreshed using the browser to retrieve the posts again.

Section 8: Application Structure and Styling

Your application must:

- use functional React components;
- use React state for values that change;
- use `useEffect()` to retrieve the initial posts;
- use `fetch()` with `async` and `await`;
- use controlled inputs;
- use props to pass data and functionality between components;
- use `map()` to display posts;
- use `filter()` where required;
- use post IDs as React keys;
- retrieve posts through the Express API;
- add posts through the Express API;
- delete posts through the Express API;
- check API responses appropriately;
- provide loading and error feedback;
- avoid unnecessary page refreshes;
- remain functional after posts are added or deleted.

A stylesheet has been supplied for the application. You may modify or extend the supplied styling if required, but styling is not the primary focus of this assignment.

The styling does not need to match a supplied example exactly, but the following parts should be visually distinguishable:

- page heading;
- add-post form;
- search input;
- individual posts;
- buttons;
- loading message;
- error messages;
- empty-post message.

Use reusable components where appropriate and avoid duplicating JSX.

Expected Application Behaviour

When the React application first loads:

- a loading message is displayed;
- React sends a request to the Express API;
- the server returns the available posts;
- the returned posts are stored in React state;
- the posts are displayed using reusable components.

When a user adds a post:

- the form values are validated;
- React sends the new post to the Express API;
- the server generates an ID;
- the server returns the newly created post;
- React adds the returned post to state;
- the post appears immediately without refreshing the page.

When a user deletes a post:

- React sends the selected post ID to the Express API;
- the server removes the post;
- React removes the post from state after receiving a successful response;
- the post disappears without refreshing the page.

When a user searches:

- the existing posts are filtered in React;
- no additional API request is made;
- the displayed posts update while the user types.

When the user selects Refresh Posts:

- the application retrieves the current posts from the server again;
- appropriate loading feedback is displayed;
- the page is not manually refreshed.

Testing Your Application

Before submitting, ensure that you have tested the following:

- the Express server starts successfully;
- `GET /api/posts` returns the supplied posts;
- the React application retrieves and displays the posts;

- the loading message appears appropriately;
- a valid post can be added;
- an empty username is rejected;
- an empty caption is rejected;
- a newly added post appears without refreshing the page;
- a post can be deleted;
- the deleted post disappears without refreshing the page;
- posts can be searched by username;
- posts can be searched by caption;
- the search is case-insensitive;
- clearing the search displays all posts;
- Refresh Posts retrieves the posts again;
- appropriate error feedback is displayed when required.

Submission Files

Submit the complete assignment folder containing:

client/

server/

Do not include any `node_modules` folders.

Ensure that the required `package.json` and `package-lock.json` files are included in both projects.

Submission Instructions

Delete the `node_modules` folders from both the `client` and `server` folders before creating your ZIP file.

ZIP the complete assignment folder using a tool such as 7-Zip or WinRAR.

Name the ZIP file:

a4-{student_number}.zip

Replace `{student_number}` with your student number.

Submit the ZIP file before the deadline to avoid a late submission.